



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +



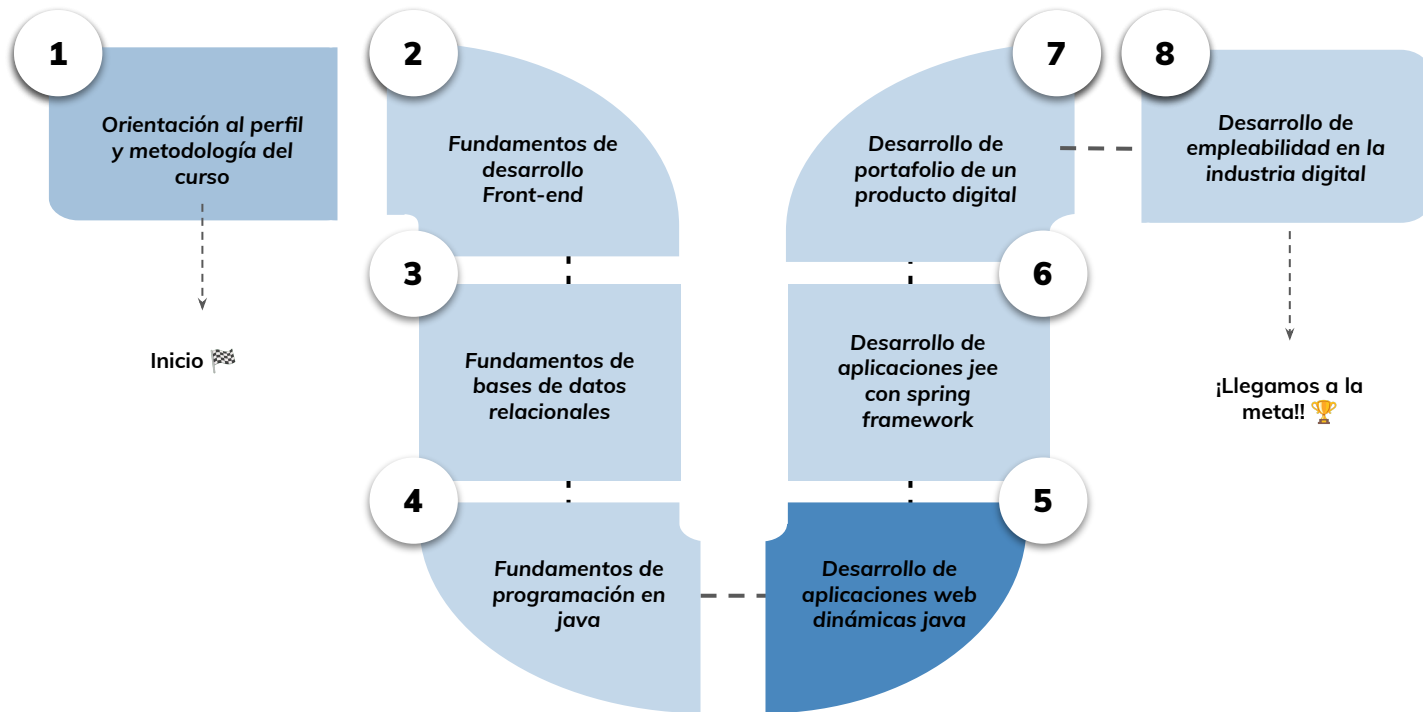
› Capa de acceso a datos

- Parte I

Aprendizaje Esperado 4: Implementar capa de acceso a datos en una aplicación web dinámica utilizando el patrón dao para resolver un problema determinado.

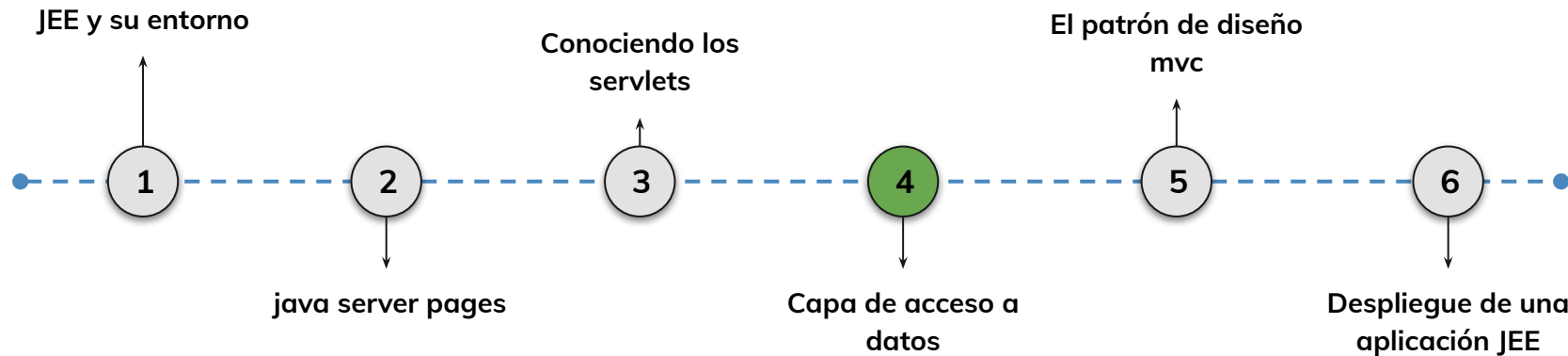
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



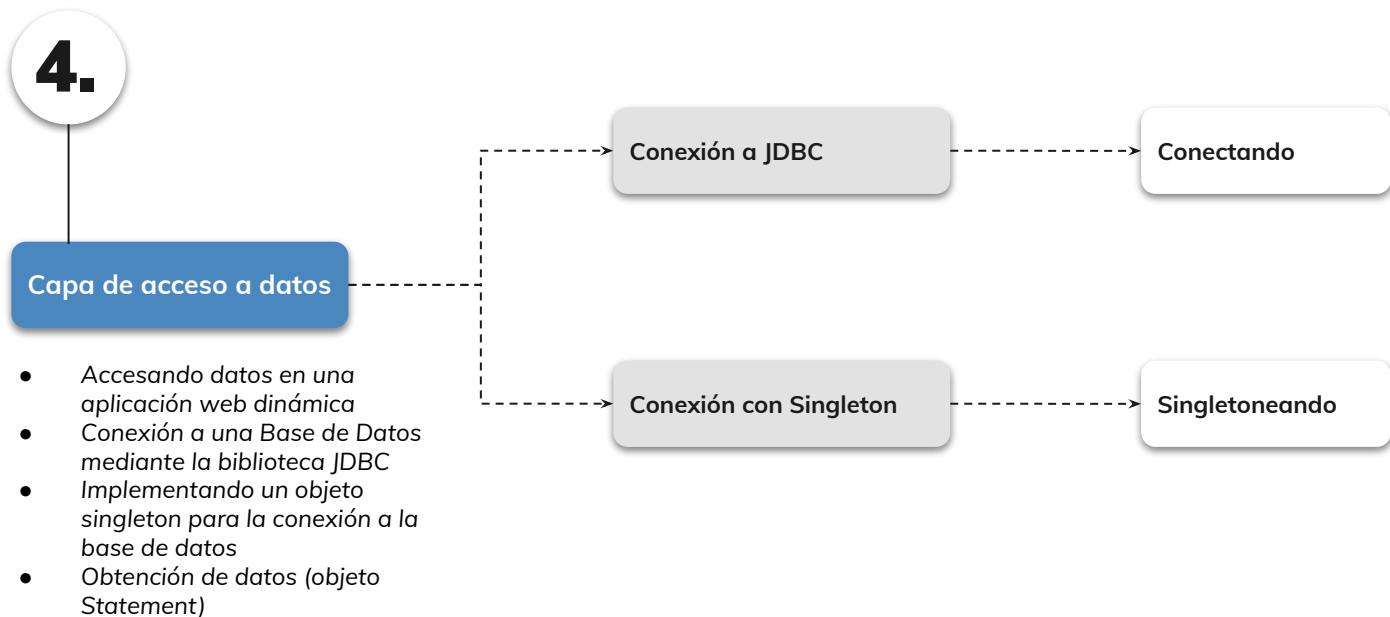
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Acceder a datos en una aplicación web dinámica
- Conectar la aplicación a una base de datos mediante JDBC
- Implementar Singleton para conectarse a la base de datos
- Obtener datos con Statement

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

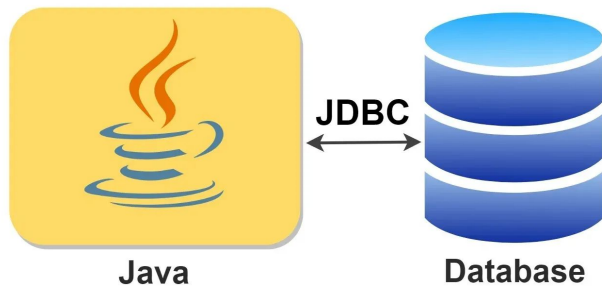
- Comprender cómo controlar los parámetros de un POST request
- Comprender cómo enviar información de un Servlet a una JSP
- Aprender a realizar validaciones en el servlet

Accesando a datos en una aplicación web dinámica

< > Accesando a datos

Java Database Connectivity (**JDBC**) **permite que los programas Java accedan a sistemas de gestión de bases de datos** . La API JDBC consiste en un conjunto de interfaces y clases escritas en el lenguaje de programación Java.

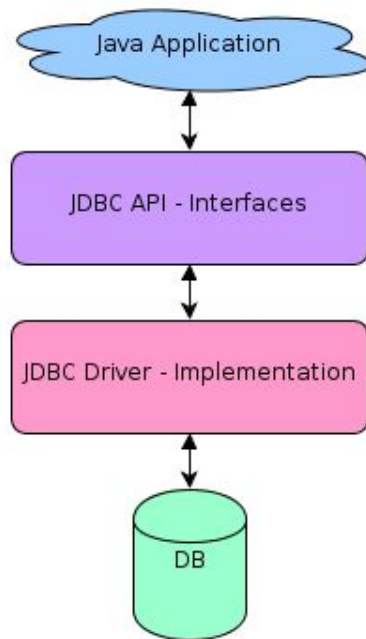
Con estas interfaces y clases estándar, los programadores pueden escribir aplicaciones que se conecten con bases de datos, envíen consultas escritas en el lenguaje de consulta estructurada (SQL) y procesen los resultados.



< > Accesando a datos

En general, hay dos **componentes principales de JDBC** a través de los cuales puede interactuar con una base de datos. Son los que se mencionan a continuación:

- **JDBC Driver Manager** : carga el driver específico de la base de datos en una aplicación para establecer una conexión con una base de datos. Se utiliza para realizar una llamada específica a la base de datos para procesar la solicitud del usuario.
- **API JDBC** : Es un conjunto de interfaces y clases, que proporciona varios métodos e interfaces para una fácil comunicación con la base de datos. Proporciona dos paquetes de la siguiente manera que contiene las plataformas java SE y java EE para exhibir capacidades WORA (write once run everything)



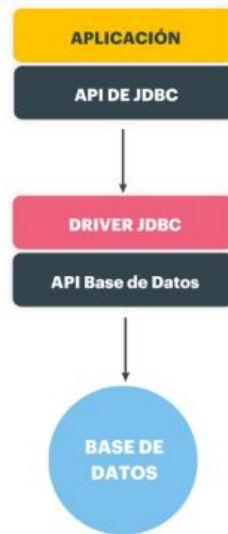
< > Accesando a datos

¿Cómo accede JDBC a la base de datos?

JDBC nos permitirá acceder a bases de datos desde Java. Para ello necesitaremos contar con un SGBD (sistema gestor de bases de datos) además de un driver específico para poder acceder a este SGBD.

La ventaja de JDBC es que nos permitirá acceder a cualquier tipo de base de datos, siempre que contemos con un driver apropiado para ella.

Como se observa en la imagen, cuando se construye una aplicación Java utilizando JDBC para el acceso a una base de datos, en la aplicación siempre se utiliza la API estándar de JDBC, y la implementación concreta de la base de datos será transparente para el usuario.



< > Accesando a datos

Componentes de la API JDBC

- **Driver** : Es el enlace que maneja toda la comunicación con la base de datos. Normalmente, una vez que se carga el controlador, el desarrollador no necesita llamarlo explícitamente.
- **Connection** : Es una interfaz con todos los métodos para contactar una base de datos. El objeto de conexión representa el contexto de comunicación, es decir, toda la comunicación con la base de datos es sólo a través del objeto de Connection.
- **Statement** : Encapsula una instrucción SQL que se pasa a la base de datos para ser analizada, compilada, planificada y ejecutada.
- **ResultSet** : Los ResultSet representan un conjunto de filas recuperadas debido a la ejecución de una consulta.



Conexión a una Base de Datos mediante JDBC

< > Conexión a una DB con JDBC

Para comunicarnos con una base de datos utilizando JDBC, se debe en primer lugar establecer una conexión con la base de datos a través del driver JDBC apropiado. El API JDBC especifica la conexión en la interfaz `java.sql.Connection`.

La clase DriverManager permite obtener objetos `Connection` con la base de datos.

Para conectarse es necesario proporcionar :

- **URL de conexión** , que incluye:
 - Nombre del host donde está la base de datos.
 - Nombre de la base de datos a usar.
- **Nombre del usuario** en la base de datos.
- **Contraseña del usuario** en la base de datos



< > Conexión a una DB con JDBC

1- Cargar el driver

El driver se debe cargar para poder utilizarlo. Esto lo realiza el método estático `forName()` de la clase `Class`. En este caso cargamos el driver de MySQL que es la base de datos con la que trabajaremos.

```
try {  
    Class.forName("com.mysql.jdbc.Driver");  
} catch (ClassNotFoundException ex) {  
    log.error("No se encontro el Driver MySQL para JDBC.");  
}
```



< > Conexión a una DB con JDBC

2- Obtener la conexión

La conexión a la BD está encapsulada en un objeto **Connection**, y para su creación se debe proporcionar la **url** de la **BD** y el **username** y **password** para acceder a ella. El formato de la url variará según el driver que se utilice.

```
// Obtener conexión a la BD
// url será el servidor de la DB
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/nombre_DB", "usuario", "password");
```



< > Conexión a una DB con JDBC

El objeto Connection representa el contexto de una conexión con la base de datos, es decir:

- Permite obtener objetos Statement para realizar consultas SQL.
- Permite obtener metadatos acerca de la base de datos (nombres de tablas, etc.)
- Permite gestionar transacciones

El siguiente código muestra un ejemplo de conexión y obtención de datos en JDBC a una base de datos MySQL.

```
try {
    Class.forName("com.mysql.jdbc.Driver");
    String url = "jdbc:mysql://hostname/database-name";
    connection = DriverManager.getConnection(url, "user", "password");
} catch (SQLException ex) {
    connection = null;
    ex.printStackTrace();
    System.out.println("SQLException: " + ex.getMessage());
    System.out.println("SQLState: " + ex.getSQLState());
    System.out.println("VendorError: " + ex.getErrorCode());
}
```

LIVE CODING

Conectando: Vamos a crear una clase donde crearemos un método que retorne la conexión a la base de datos.

Debemos crear una DB en MySQL para poder probar la conexión.

Tiempo: 20 minutos



Ejercicio N° 1

Conectando

Conectando

Manos a la obra: 🙌

Es hora de realizar nuestra primer conexión con la base de datos! Para esto es necesario que crees una nueva database MySQL llamada wallet.

Consigna: 📝

1. Crear la BD Wallet.
2. Crear un nuevo paquete en el proyecto AlkeWallet donde guardaremos los archivos de conexión.
3. Crear una clase que manejará conexiones y que contenga el método `createConnection` que debe retornar la conexión exitosa a la BD.

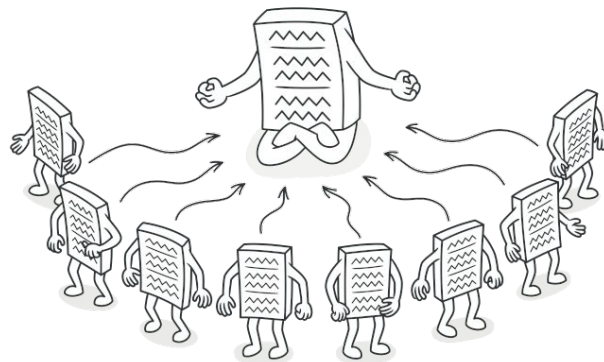
Tiempo 🕒: 30 minutos

Implementando Singleton para conectarse a la DB

< > Implementando Singleton

Singleton es un **patrón de diseño creacional** que nos permite asegurarnos de que una clase tenga **una única instancia**, a la vez que proporciona un punto de acceso global a dicha instancia.

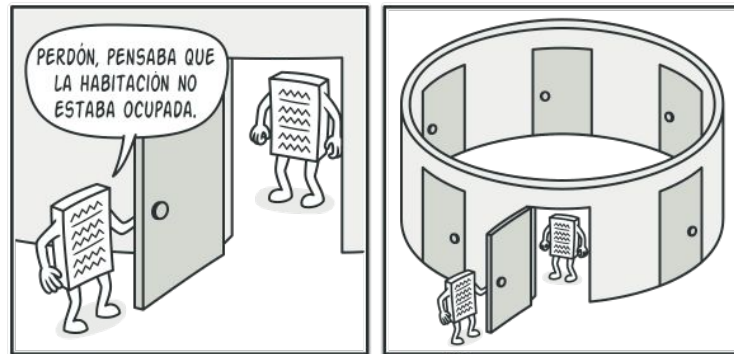
Implementar un objeto singleton para la conexión a la base de datos es una práctica común para asegurar que solo exista una instancia de la conexión y reutilizarla en toda la aplicación. Esto puede mejorar el rendimiento y la eficiencia, evitando la creación excesiva de conexiones.



< > Implementando Singleton

Para comunicarnos con una base de datos utilizando JDBC, se debe en primer lugar establecer una conexión con la base de datos a través del driver JDBC apropiado. El API JDBC especifica la conexión en la interfaz `java.sql.Connection`.

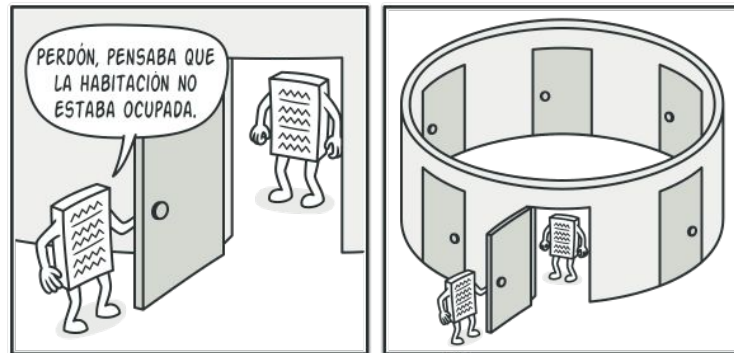
La clase `DriverManager` permite obtener objetos `Connection`.



< > Implementando Singleton

Para conectarse es necesario proporcionar:

- URL de conexión, que incluye:
 - Nombre del host donde está la base de datos.
 - Nombre de la base de datos a usar.
- Nombre del usuario en la base de datos.
- Contraseña del usuario en la base de datos.



< > Implementando Singleton

Como vimos la clase anterior, una vez cargado el driver se debe establecer la conexión con la BD. Para ello se utiliza el siguiente método:

`Connection` conexion = `DriverManager.getConnection`(url, login, password);

```
protected void conectarBase(){
    try {
        //registrar el Driver de MySQL
        Class.forName("com.mysql.jdbc.Driver");

        //URL de la DB
        String urlDataBase = "jdbc:mysql://localhost:3306/"+database+"?useSSL=false";

        //Establecemos la conexion!
        Connection conexion = DriverManager.getConnection(urlDataBase, user, password);
    } catch (Exception e) {
        System.out.println("Error en Conectar Base");
    }
}
```



< > Implementando Singleton

El objeto Connection representa el contexto de una conexión con la base de datos, es decir:

- Permite obtener **objetos Statement para realizar consultas SQL** .
- Permite obtener metadatos acerca de la base de datos (nombres de tablas, etc.)
- Permite gestionar transacciones.

```
protected void conectarBase(){
    try {
        //registrar el Driver de MySQL
        Class.forName("com.mysql.jdbc.Driver");

        //URL de la DB
        String urlDataBase = "jdbc:mysql://localhost:3306/"+database+"?useSSL=false";

        //Establecemos la conexion!
        Connection conexion = DriverManager.getConnection(urlDataBase, user, password);

    } catch (Exception e) {
        System.out.println("Error en Conectar Base");
    }
}
```



< > Implementando Singleton

Veamos el ejemplo respetando el patrón Singleton:

Como podemos ver, creamos los objetos como atributos protegidos y las variables finales para que la clase EjemploSingleton tenga todas las instancias de clase necesarias.

```
public class EjemploSingleton {
    //OBJETO CONNECTION ---> Encargado de INICIAR - TENER - MANTENER la conexion
    protected Connection conexion = null;
    //OBJETO RESULTSET ---> Guardar todos los datos que llegan de la DB (las filas de la consulta)
    protected ResultSet resultado = null;
    //OBJETO STATEMENT ---> "Tiene" las consultas... es donde generamos las sentencias a ejecutar!
    protected Statement stmt = null;

    private final String user = "root";
    private final String password= "root";
    private final String database = "nombre_database";

    protected void conectarBase(){
        try {
            //registrar el Driver de MySQL
            Class.forName("com.mysql.jdbc.Driver");

            //URL de la DB
            String urlDataBase = "jdbc:mysql://localhost:3306/"+database+"?useSSL=false";

            //Establecemos la conexion!
            conexion = DriverManager.getConnection(urlDataBase, user, password);

        } catch (Exception e) {
            System.out.println("Error en Conectar Base");
        }
    }
}
```

< > Implementando Singleton

De manera consecutiva a la creación de una conexión, **es necesario desconectar la base**. Para esto, vamos a crear un método que verifique si hay alguna transacción abierta y en ese caso que la cierre.

```
protected void desconectarBase(){
    try {
        if (resultado!=null) {
            resultado.close();
        }
        if (stmt!=null) {
            stmt.close();
        }
        if (conexion!=null) {
            conexion.close();
        }
    } catch (Exception e) {
        System.out.println("Error en Desconectar Base!");
    }
}
```



< > Implementando Singleton

Una vez que tengamos la conexión con el objeto Connection encapsulado en un objeto singleton, la vamos a usar para crear un objeto **Statement**, **este objeto recibe la consulta para ejecutarla y enviársela a la base de datos** .

La información que recibimos de la base de datos va a ser capturada por el objeto ResultSet para después poder mostrar la información.



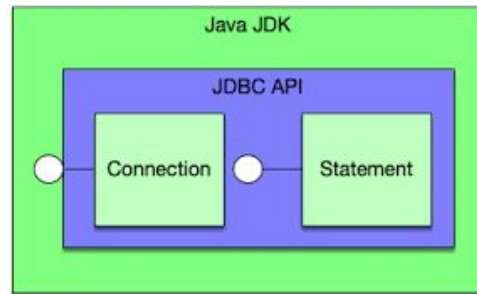
Obtención de datos: Statement

< > Statement

Una vez realizada la conexión a la BD, se puede utilizar para crear sentencias. Estas sentencias están encapsuladas en la clase **Statement**, y se pueden crear de la siguiente forma:

```
Statement stmt= conexion.createStatement();
```

- Los objetos **Statement** **se obtienen a partir de un objeto Connection** .
- **Permiten realizar consultas SQL** en la base de datos.



< > Statement

Una vez obtenido el objeto Statement, se puede ejecutar sentencias utilizando su método `executeQuery()` al que se proporciona una cadena con la sentencia SQL que se quiere ejecutar:

```
stmt.executeQuery(sentenciaSQL);
```

Estas sentencias pueden utilizarse para consultas a la base de datos.

```
// Crear sentencia SQL
String sql = "SELECT * FROM products";

// Preparar sentencia
Statement stmt = conexion.createStatement();

// Ejecutar consulta
stmt.executeQuery(sql);
```



< > Statement

Las sentencias SQL se van a escribir entre comillas dobles ya que el objeto Statement recibe String, como dato para las sentencias.

```
String sentenciaSQL = "SELECT nombre, apellido FROM alumnos";
```

```
// Crear sentencia SQL
String sql = "SELECT * FROM products";

// Preparar sentencia
Statement stmt = conexion.createStatement();

// Ejecutar consulta
stmt.executeQuery(sql);
```



< > Statement

Como podemos ver las sentencias van a tener la **misma sintaxis que en MySQL**, la única **diferencia** se va a presentar a la hora de trabajar con datos de tipo **String** y de tipo **Date**.

Como estos datos suelen ir en comillas dobles en Java y en SQL, y nuestra sentencia ya está entre comillas dobles, **debemos poner los datos entre comillas simples** para diferenciarlos.

```
// sentencia SQL STRING
String sqlString = "SELECT * FROM products WHERE nombre = 'notebook';

// sentencia SQL DATE
String sqlDate = "SELECT * FROM products WHERE vencimiento = '18-12-2022';
```



LIVE CODING

¡Singleton y Statement!

Vamos a crear una Clase Java que tenga encapsulada toda la conexión con el patrón de diseño Singleton. Esta clase debe tener métodos para conectarse, desconectarse y realizar una consulta (sentencia).

Tiempo: 20 minutos



Ejercicio N° 1

Singletoneando

Singletoneando

Manos a la obra: 🙌

Vamos a modificar el código que venimos trabajando para que respete el patrón Singleton.

Te invitamos a que generes algunos registros en tu BD Wallet para poder probar las consultas.

Consigna: 🛠️

1. Crear una clase que respete el patrón Singleton de conexión.
2. Adecuar el método `createConnection` al patrón de diseño Singleton. Además, debes crear el método `closeConnection` (para cerrar las conexiones existentes).

Tiempo 🕒: 30 minutos



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Conocer el acceso a datos desde la aplicación
- ✓ Aprender a configurar JDBC para conectarse a una base de datos
- ✓ Comprender la implementación del patrón de diseño Singleton para la conexión.
- ✓ Aprender la implementación del objeto Statement.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

